Search & Recommendations

Purpose

We provide APIs for product search, filters, facets, autocomplete and recommendations. Storefront owns customer UI/UX. The exercise uses a small mock-data search task; the full system is not implemented.

Structure

Within src/features/search-recommendations/: schemas/ holds Zod validation; types/ holds data contracts; server/services/ holds search/ranking; server/repositories/ holds data adapters; tests/ holds Vitest tests. These remain scaffolds. server/index.ts is server-only and index.ts is client-safe. Planned thin HTTP routes live under src/app/api/v1/.

Interfaces

Incoming ownership is disputed between Admin and Marketplace; a joint review is open. Do not assume their IDs refer to the same record. Admin proposes { article_id: string, name: string, description?: string, category_id: string, tags: string[], is_published: boolean }. Marketplace proposes { offerId: string, productId: string, sellerId: string, name: string, description: string, category: string, attributes: object, imageUrls: string[], priceMinor: number, currency: string, availability: string, status: string, updatedAt: string }.

For new mocks, use Marketplace's proposal, with EUR prices and string-valued attributes as local assumptions. Its stated availability values are in_stock/out_of_stock; only status=active is public. pending/rejected/blocked/removed are excluded. Marketplace proposes GET /offers?updatedSince=<timestamp>, including blocked/removed changes; final envelope, access, pagination and timestamp rules are pending. One seller per product does not prove one offer per product.

Storefront-facing proposals: GET /api/v1/search with q, category, minPriceMinor, maxPriceMinor, page, pageSize returns { data: { items, total, page, pageSize } }; GET /api/v1/recommendations/similar with productId, limit returns { data: { items } }. Items contain offerId, productId, sellerId, name, category, imageUrls, priceMinor, currency, availability. Result cardinality and Storefront acceptance remain pending. Initial search: trimmed case-insensitive name substring, exact category, inclusive AND price bounds; page=1, pageSize=20, maximum 100. Similarity uses category/attributes, not purchase history.

Conventions

Strict TypeScript; kebab-case files, camelCase functions, PascalCase types. Validate inputs with Zod; keep routes thin and logic testable. Errors use { error: { code, message } }: 400 INVALID_INPUT, 404 PRODUCT_NOT_FOUND for unavailable recommendation source, 503 CATALOG_UNAVAILABLE for source failure. Use Vitest for normal, invalid, boundary and visibility cases. Cross-team access uses HTTP contracts, not internal imports or direct table reads.

Pitfalls

Product/article likely names the same concept; offer adds seller terms, but shared ownership/IDs are not agreed. category is not category_id; attributes are not tags; status is not is_published. Marketplace priceMinor means integer cents including VAT before Marketing discounts. Stock and publication are separate. Co-purchase recommendations need an unassigned purchase-data source. Webhooks, JWT permissions, internal search and reindexing are proposals, not confirmed interfaces. The recorded experiment used an older frozen 663-token context; never rewrite that evidence.

Context measurement: 690 tokens | tiktoken 0.14.0 | o200k_base